

# TRELLIS — Geometry Generation vs. Texturing

2 August 2026

## Contents

|                                                        |          |
|--------------------------------------------------------|----------|
| <b>Summary</b>                                         | <b>1</b> |
| <b>1. TRELLIS v1 — one latent, several decoders</b>    | <b>2</b> |
| The representation . . . . .                           | 2        |
| The pipeline <b>[verified]</b> . . . . .               | 2        |
| What this means practically . . . . .                  | 2        |
| <b>2. TRELLIS.2 — geometry and texture split apart</b> | <b>4</b> |
| What changed in the representation . . . . .           | 4        |
| The texturing pipeline <b>[verified]</b> . . . . .     | 4        |
| The two modes, side by side . . . . .                  | 5        |
| <b>3. Why this matters for the proposal</b>            | <b>6</b> |
| <b>4. What to read, in order</b>                       | <b>7</b> |
| One caution on secondary sources . . . . .             | 7        |
| <b>Appendix — the one-paragraph version</b>            | <b>8</b> |

## Summary

**TRELLIS v1 has no separate texturing mode.** One latent encodes geometry *and* appearance together; you choose an output format at decode time. There is no way to hand it an existing mesh and ask only for texture.

**TRELLIS.2 adds exactly that.** It ships a dedicated `Trellis2TexturingPipeline` with its own encoder, flow model and decoder, which takes an **existing shape plus a reference image** and generates **PBR texture conditioned on that shape**. Geometry and texture live in two separate latent spaces with separate normalisation statistics.

That distinction is the answer to “how does TRELLIS differ in texturing vs geometry mode”, and it is directly relevant to our work: we are doing *temporal* texturing on a fixed mesh, which is the TRELLIS.2 texturing formulation extended over time.

Everything below marked **[verified]** was read from the model configuration files and source code on our cluster, not from documentation.

# 1. TRELLIS v1 — one latent, several decoders

**Paper:** Xiang et al., *Structured 3D Latents for Scalable and Versatile 3D Generation*, **CVPR 2025 (Spotlight)**, arXiv:2412.01506.

## The representation

The contribution is **SLat** (Structured LATents): a sparsely-populated 3D voxel grid where each *active* voxel — one that intersects the object’s surface — carries a local latent vector. Those latents are produced by encoding **dense multiview visual features from a vision foundation model (DINOv2)** onto the voxels.

The paper’s own framing is the important part for the grant:

SLat “comprehensively captur[es] both structural (geometry) and textural (appearance) information while maintaining flexibility during decoding.”

**Geometry and appearance are deliberately entangled in a single latent.** Flexibility comes at *decode* time, not at generation time.

## The pipeline [verified]

From pipeline.json in microsoft/TRELLIS-image-large:

TrellisImageTo3DPipeline

|                             |                            |                                    |
|-----------------------------|----------------------------|------------------------------------|
| sparse_structure_flow_model | ss_flow_img_dit_L_16l8     | 16 <sup>3</sup> latent → occupancy |
| sparse_structure_decoder    | ss_dec_conv3d_16l8         | → which voxels exist               |
| slat_flow_model             | slat_flow_img_dit_L_64l8p2 | → 8 numbers per voxel              |
| slat_decoder_gs             | → 3D Gaussians             |                                    |
| slat_decoder_rf             | → radiance field           |                                    |
| slat_decoder_mesh           | → triangle mesh            |                                    |

Two generative stages, then a **choice of three decoders reading the same SLat**.

| stage                 | what it decides                                                             |
|-----------------------|-----------------------------------------------------------------------------|
| sparse structure flow | <i>which</i> voxels are occupied — coarse geometry                          |
| SLat flow             | <i>what</i> is in each voxel — fine geometry <b>and</b> appearance, jointly |
| decoder               | which output format you want                                                |

Scale: up to 2B parameters, trained on 500K 3D assets.

## What this means practically

- You cannot give TRELLIS v1 a mesh and ask for texture. There is no entry point for it.
- Geometry and appearance cannot be varied independently at generation time — they come out of the same flow model in the same latent.
- The three decoders are three *readings* of one latent, not three modes.

**Our project works at this decode boundary.** The mesh decoder’s final layer emits 101 numbers per voxel, and their roles are fixed and separable **[verified, trellis/representations/mesh/cube2mesh.py]**:

| channels | role                                     |
|----------|------------------------------------------|
| [0:8]    | sdf — the surface’s inside/outside field |
| [8:32]   | deform — corner displacements            |
| [32:53]  | FlexiCubes weights — topology            |
| [53:101] | colour — 24 albedo + 24 shading normal   |

Vertex positions are computed from sdf and deform **only**. That is what lets us edit [53:101] and *prove* the geometry did not move — measured across 150 frames and a full 360° orbit: zero vertex-count differences, zero-pixel silhouette difference.

## 2. TRELIS.2 — geometry and texture split apart

**Reference:** Xiang et al., *Native and Compact Structured Latents for 3D Generation*, tech report, 2025, arXiv:2512.14692. Model: microsoft/TRELIS.2-4B, MIT licence.

### What changed in the representation

|                    | TRELIS v1                            | TRELIS.2                                                                     |
|--------------------|--------------------------------------|------------------------------------------------------------------------------|
| representation     | SLat on surface-intersecting voxels  | <b>O-Voxel</b> — a <i>field-free</i> sparse voxel structure                  |
| surface extraction | iso-surface field (SDF + FlexiCubes) | none — no field to extract from                                              |
| compression        | —                                    | 16× spatial downsampling; a $1024^3$ asset → <b>~9.6K latent tokens</b>      |
| appearance         | RGB                                  | <b>PBR</b> — base colour, roughness, metallic, <b>opacity</b> (translucency) |
| output resolution  | 256 <sup>3</sup> mesh grid           | up to <b>1536<sup>3</sup></b>                                                |
| image encoder      | DINOv2 <b>[verified]</b>             | <b>DINOv3</b> dinov3-vitl16-pretrain-lvd1689m <b>[verified]</b>              |
| background removal | rembg                                | <b>BiRefNet / RMBG-2.0 [verified]</b>                                        |
| scale              | ~2B                                  | <b>4B</b>                                                                    |

Dropping the iso-surface field is the structural change. TRELIS v1’s mesh output goes through FlexiCubes, which converts an sdf into triangles; TRELIS.2’s O-Voxel has no field, so there is nothing to iso-surface.

### The texturing pipeline [verified]

microsoft/TRELIS.2-4B ships a file named `texturing_pipeline.json`. Its contents:

Trellis2TexturingPipeline

|                          |                             |                               |
|--------------------------|-----------------------------|-------------------------------|
| shape_slat_encoder       | shape_enc_next_dc_f16c32    | existing shape → shape latent |
| tex_slat_flow_model_512  | slat_flow_imgshape2tex_1.3B | (image, shape) → texture      |
| tex_slat_flow_model_1024 | slat_flow_imgshape2tex_1.3B | same, higher resolution       |
| tex_slat_decoder         | tex_dec_next_dc_f16c32      | texture latent → PBR texture  |
| image_cond_model         | DinoV3FeatureExtractor      |                               |

The flow model’s own name states the contract: **imgshape2tex** — image + shape → texture.

The channel counts show mechanically how shape conditions texture **[verified, slat\_flow\_imgshape2tex\_dit\_1\_3B\_512]**

|                |      |                                                               |
|----------------|------|---------------------------------------------------------------|
| in_channels    | 64   | = 32 (noisy texture latent) + 32 (shape latent), concatenated |
| out_channels   | 32   | texture only – the shape is never predicted                   |
| model_channels | 1536 |                                                               |
| num_blocks     | 30   |                                                               |

cond\_channels    1024        DINOv3 image tokens, entering by cross-attention  
 resolution       32  
 sampling steps   12

**The shape enters as concatenated channels, not as something the model generates.** It is a fixed condition. The model can only produce texture.

And the two latent spaces are genuinely distinct — the pipeline carries **separate 32-dimensional normalisation statistics** for shape\_slat\_normalization and tex\_slat\_normalization, with different means and standard deviations. They are not two views of one space.

### The two modes, side by side

|                   | <b>geometry mode</b> (image → 3D) | <b>texturing mode</b> (shape + image → texture) |
|-------------------|-----------------------------------|-------------------------------------------------|
| pipeline          | Trellis2ImageTo3DPipeline         | Trellis2TexturingPipeline                       |
| input             | one image                         | <b>an existing mesh</b> + a reference image     |
| generates         | geometry <b>and</b> texture       | texture <b>only</b>                             |
| geometry          | produced by the model             | <b>given, and held fixed</b>                    |
| conditioning      | image                             | image <b>and</b> shape latent                   |
| in / out channels | —                                 | 64 in / <b>32 out</b>                           |

In v1 the second column does not exist.

### 3. Why this matters for the proposal

**The field has moved from “generate a textured object” to “texture a given object.”** TRELLIS v1 (CVPR 2025) fuses geometry and appearance in one latent by design; TRELLIS.2 (late 2025) separates them and ships a dedicated shape-conditioned texturing pipeline. That separation is what makes texture an independently controllable variable.

**Our work is the temporal extension of exactly that formulation.** TRELLIS.2 textures a fixed shape from one image. We texture a fixed shape from **a video** — one mesh, a *series* of textures, geometry provably unchanged. The natural framing is: *shape- conditioned texturing exists for a single image; the temporal case is open.*

**Two honest qualifications** worth keeping in the text:

1. Our current results are built on **TRELLIS v1**, not TRELLIS.2. We obtain the geometry/texture separation ourselves, by splicing the mesh decoder’s frozen geometry channels with adapted colour channels — a guarantee we verify per frame rather than inherit from the architecture.
2. TRELLIS.2’s texturing pipeline is **single-image**. Nothing in it addresses temporal consistency across frames, which is the problem we are working on.

## 4. What to read, in order

**1. The TRELLIS v1 paper — read §3 (SLat) and the decoder section.** Xiang, Lv, Xu, Deng, Wang, Zhang, Chen, Tong, Yang. *Structured 3D Latents for Scalable and Versatile 3D Generation*. CVPR 2025 Spotlight. <https://arxiv.org/abs/2412.01506> · project page <https://microsoft.github.io/TRELLIS/>

Gives you the SLat definition and the explicit claim that one latent carries geometry and appearance together. This is the sentence to cite for “v1 fuses them.”

**2. The TRELLIS.2 tech report — for O-Voxel, PBR, and the texturing pipeline.** Xiang, Chen, Xu, Wang, Lv, Deng, Zhu, Dong, Zhao, Yuan, Yang. *Native and Compact Structured Latents for 3D Generation*. Tech report, 2025. <https://arxiv.org/abs/2512.14692> · project page <https://microsoft.github.io/TRELLIS.2/> · model card <https://huggingface.co/microsoft/TRELLIS.2-4B>

The tech report is the citable source for O-Voxel, the 9.6K-token compression and PBR with opacity. Note it is a **tech report, not peer-reviewed** — say so if the venue matters.

**3. texturing\_pipeline.json in the model repo — the primary source for the split.** [https://huggingface.co/microsoft/TRELLIS.2-4B/blob/main/texturing\\_pipeline.json](https://huggingface.co/microsoft/TRELLIS.2-4B/blob/main/texturing_pipeline.json)

Four model entries and two normalisation blocks. Two minutes to read and it settles the geometry/texture question more definitively than any prose. This is where every **[verified]** claim above comes from.

**4. Code, if implementation detail is needed.** <https://github.com/microsoft/TRELLIS> (v1) · <https://github.com/microsoft/TRELLIS.2>

For v1, `trellis/representations/mesh/cube2mesh.py` is the file that shows geometry and colour occupying disjoint channel ranges.

**5. Adjacent work worth a sentence, if the proposal needs positioning.** Jin, Xie, Zheng, Wang, Bao, Huo. *Fuse3D: Generating 3D Assets Controlled by Multi-Image Fusion*. SIGGRAPH Asia 2025. <https://arxiv.org/abs/2602.17040>

Controls TRELLIS **training-free** by scaling cross-attention logits in the SLat flow. Relevant because it demonstrates that TRELLIS’s cross-attention already encodes a usable 2D↔3D correspondence — a lever for control that needs no fine-tuning.

### One caution on secondary sources

DeepWiki pages for these repositories are **auto-generated from source code**. Useful for orientation, not citable. Everything in this document is traced either to the papers above or to configuration files read directly from the model weights on our cluster.

## Appendix — the one-paragraph version

TRELLIS (Xiang et al., CVPR 2025) represents a 3D asset as *Structured Latents*: a sparse voxel grid whose active voxels carry local latents encoding both geometry and appearance. Because the two are fused in a single latent, the model generates them jointly and offers flexibility only at decode time, through interchangeable Gaussian, radiance-field and mesh decoders — there is no facility for texturing an existing shape. TRELLIS.2 (Xiang et al., tech report 2025) restructures this: it replaces the iso-surface field with a field-free O-Voxel representation and, critically, factors generation into a shape latent and a texture latent with separate encoders, decoders and normalisation. This enables a dedicated shape-conditioned texturing pipeline in which an existing mesh is encoded and held fixed while a 1.3B flow model generates PBR texture conditioned on both that shape and a reference image. Texture thereby becomes an independently controllable variable — but only for a single static image. Extending shape-conditioned texturing to the temporal setting, where one mesh must carry a coherent sequence of textures, remains open.